

# The 'sopra-listings' Package

Documentation for the 'sopra-listings' package | Version v1.0.23

1. October 2020

Florian Sihler (florian.sihler@uni-ulm.de)

## 1 General

### 1.1 Why, wherefore, what for?

This  $\text{\LaTeX}$  package was written as part of the Sopra project in the winter semester 2019 and summer semester 2020 and serves as a foundation for highlighting the code of *Team 20*. This documentation was created together with the `sopra-base.cls` class as well as the package `sopra-documentation.sty`.

To visualize the individual code snippets, the package itself is used, which should also be embedded here: . This package needs neither `shell-escape` nor `minted` to work. It builds on `listings`!

### 1.2 Dependencies

This package includes the following packages:

- |                                                     |                         |                                                                            |
|-----------------------------------------------------|-------------------------|----------------------------------------------------------------------------|
| ▪ <code>fontenc</code> <sup>(T1, encoding)</sup>    | ▪ <code>etoolbox</code> | ▪ <code>listingsutf8</code> <sup>(<code>&lt;&lt;try&gt;&gt;</code>)</sup>  |
| ▪ <code>inputenc</code> <sup>(utf8, encoding)</sup> | ▪ <code>xcolor</code>   | ▪ <code>listings</code> <sup>(<code>&lt;&lt;fallback&gt;&gt;</code>)</sup> |
| ▪ <code>needspace</code> <sup>(guardspace)</sup>    | ▪ <code>pgfkeys</code>  | ▪ <code>accsupp</code> <sup>(<code>&lt;&lt;try&gt;&gt;</code>)</sup>       |

All these packages should be part of a common  $\text{\LaTeX}$  distribution. Note: “`<<try>>`” indicates that the package does not need to be present, but may offer additional possibilities. For example, `accsupp` allows the line numbers to be made “uncopyable”, making it easier to work with the listings, and `listings` is only loaded when `listingsutf8` cannot be found.

Currently, a few things like options and further environments are still planned.

### 1.3 Installation

The package is not distributed as a `.dtx` file, which is why the following options are available:

- The package can be placed in the same directory as the document. In this case, the include instruction is:  
`\usepackage{sopra-listings}`
- The package can be placed in a subdirectory/a directory shipped with the document. In this case, it is specified via the (relative) path:  
`\usepackage{./My/Path/to/sopra-listings}`
- The package can be placed (via a symlink or similar) in a `texmf` tree of your own. For example, on Linux when using `texlive`, the package can be placed here: `~/texmf/tex/latex/`. The directory can be created and afterwards updated using `texhash ~/texmf`. Now the package can be used like any other installed package:

```
\usepackage{sopra-listings}
```

**Important note:** This package comes with language configurations located in the subfolder “Languages”. The languages are only available if they are placed with the package (following one of the options just mentioned) or if they are located in the same directory as the  $\LaTeX 2_{\epsilon}$  file. Should this not be desired or possible, `\solLanguageSearchPath`<sup>→ p. 6</sup> *must* be adjusted! Everyone is free to disable the loading of the languages via `noloadlangs` and to use their own language definitions or languages. However, for this package to make sense, the `\RegisterLanguage`<sup>→ p. 7</sup> macro should be used.

## 1.4 Further Notes

Version v1.0.0:

`\ttfamily` is modified so that the “typewriter” font here is also identical to the one used generally.

Version v1.0.10:

The command `\lstfs`<sup>→ p. 8</sup> was added to adjust the font sizes of listings.

Version v1.0.12:

The package option `cpalette` was added along with its counter-option.

Version v1.0.13:

The commands `\lstcolorlet`<sup>→ p. 9</sup> and `\lstcolordef`<sup>→ p. 9</sup> were added for easy color manipulation.

Version v1.0.14:

Spacing for literates fixed. The correct number of blank fields should now also be kept when replacing digits in inline listings

Version v1.0.15:

With `guardspace` (`noguardspace`)<sup>→ p. 3</sup> a (vertical) space-guarding option was added.

Version v1.0.17:

With `autogobble`, leading blank fields can now be removed.

Version v1.0.18:

Now also secures the last line of a listing in case of `guardspace` (`noguardspace`)<sup>→ p. 3</sup>.

Version v1.0.19:

Italic font for the typewriter typeface is now applied more reliably. A few unused convenience functions were removed.

Version v1.0.20:

The `autogobble` function was removed again, and the package structure was also reworked internally.

Version v1.0.21:

Some internal code restructurings were carried out.

Version v1.0.22:

Support for `nofakeminted` (`fakeminted`)<sup>→ p. 4</sup> and `noextendednums` (`extendednums`)<sup>→ p. 4</sup>.

Version v1.0.23:

Support for `\solblacklistlinenumbers` <sup>→ p. 9</sup>, `\solblacklistisghost` <sup>→ p. 10</sup> and `\solblacklistispresent` <sup>→ p. 10</sup>.

## 1.5 Accepted Parameters

Like most packages, this one accepts arguments. For arguments with a “counter” option, the one active by default is written first and the other in parentheses. So the following is implicitly:

```
\usepackage[noencoding,hlnumbers,loadlangs,guardspace]{sopra-listings}
```

invoked. While with:

```
\usepackage[noloadlangs]{sopra-listings}
```

the automatic loading of certain languages is prevented.

- ▷ `noencoding`      (`encoding`)

encoding loads the packages needed for umlauts. This option can be used if you do not want to load them separately, or simply want to be on the safe side.

- ▷ `hlnumbers`      (`nolnnumbers`)

The package automatically tries to mark numbers, and only where it is actually a number. This procedure works well, but can - should it be perceived as disturbing, or turn out to misbehave after all - be disabled via `nolnnumbers`.

- ▷ `defaultmode`    (`print`)

These functions work analogously to the like-named counterparts in `sopra-base.cls`. If `print` is chosen, an attempt is made to save as much ink as possible (no background for code, ...) and furthermore certain key terms are printed bold/italic in order to retain (as good as possible) distinguishability even in a single-color printout.

- ▷ `loadlangs`      (`noloadlangs`)

Loads the language java via `\solLoadLanguage` <sup>→ p. 7</sup> (see Section A.1 (Predefined Languages) for more information). This can be disabled via `noloadlangs`. Note: The languages are only loaded if the configuration files are found. See `\solLoadLanguage` <sup>→ p. 7</sup> for this.

- ▷ `nocpalette`      (`cpalette`)

Offers support for <https://github.com/Eagleoutlce/color-palettes>. In this case the code highlighting is automatically adapted to the current palette and is also updated from then on whenever the palette changes.

- ▷ `noupshape`      (`upshape`)

Adds `\upshape` to the style. This ensures that listings no longer adapt to italic text.

- ▷ `guardspace`    (`noguardspace`)

With `guardspace`, the listing is each time also guaranteed to be allowed to set the next line. This can cause a page break to occur (if less than one line of the listing still fits on the page). The purpose of this option is to avoid initial stripes with a background color. See `\solguard` <sup>→ p. 9</sup>.

- ▷ `nonuminpar`      (`numinpar`)

Short form for `\solSetLeftMargin`<sup>→ p. 8</sup> with the value “2em”, can thus be used to automatically indent listings and thereby prevent the numbers from protruding beyond the document margin.

- ▷ `defaultfont`      (`nodefaultfont`)

If the option is set, the typewriter font is changed to “AnonymousPro”, otherwise not.

- ▷ `nofakeminted`      (`fakeminted`)

If the option is set, a `env@minted` environment is created which allows this package to be used like `env@minted`. In this case it reads:

```
\begin{minted}{java}
public static void main(String[] args) { }
\end{minted}
```

With `\solmintedmap`<sup>→ p. 10</sup> a language mapping can be created. For example, java is automatically mapped to the internal language lJava. Using `\solsetmintedstyle`<sup>→ p. 11</sup> you can choose which of the internal styles should be used for the listing. Available are `default`, `nonumber`, `plain` and `plain number`.

- ▷ `noextendednums`      (`extendednums`)

If the option is set, numbers are also highlighted that internally use underscores or the classic `0x`, `0b` or `0o` prefixes.

- ▷ `inlinesize`      (`noinlinesize`)

If the option is set, the font size in the inline commands such as `\bjava` is not adjusted (see also `\lstfs`<sup>→ p. 8</sup> with an asterisk for this). Within the document, this configuration can be enabled at any time with `\soladaptinline` and disabled with `\sollockinline`.

## 1.6 Colors

This package defines a few colors using `\colorlet`. If `sopra-base` is detected as loaded, an attempt is made to set matching colors. *All of the following colors carry the prefix `\sol@colors@`:* `border`, `background`, `lst@keywordA`, `lst@keywordB`, `lst@keywordC`, `lst@numbers`, `lst@literals`, `lst@comments`, `lst@highlight`, `lst@linenumbers` and lastly `lst@command`, where the meaning of each color should be apparent from its name (every language is free to use the styles discussed below, which build on these colors, (in any way it wishes)). For example, the color of comments can be changed as follows (`\lstcolorlet`<sup>→ p. 9</sup>):

```
\lstcolorlet{comments}{orange}
```

We get (exemplarily in Java, first the default, then with the changed color):

```
1 // Before reassigning the color
2 public static void main(...){ /* ... */ }
```

```
1 // After reassigning the color
2 public static void main(...){ /* ... */ }
```

It is possible to use any color (following `xcolor`) (see also `\lstcolordef`<sup>→ p. 9</sup>).

## 1.7 Styles

To highlight the individual segments in the listing, styles derived from the colors are defined, which can likewise be freely changed. Developer note: the styles should, in the case of a complete redefinition, be defined with the help of the command `\sol@list@define@styles`. Otherwise, the prefix to be used below is stored in `\sol@lst@style@prefix`.

All commands listed below carry (unless modified) the prefix `\sol@style@lst@` and can be queried (without this prefix) via `\solGet`<sup>→P. 8</sup> or `\solGetStyle`<sup>→P. 8</sup> respectively. Individual languages can require and add further commands and styles, but this should only be done in case of emergency (for example, the language `Latex` adds the style command. For more information, see: Section A.1 (Predefined Languages))

There are the following styles: `keywordA`, `keywordB`, `keywordC`, `keywordD`, `numbers`, `linenumbers`, `literals`, `comments`, `highlight`, `command` and `basic`, the latter being prepended to every style. Redefining a style should not be done lightly and therefore requires a bit more effort. Here is an example:

```
{\makeatletter
  \sol@list@define@styles{%
    {comments: \color{sol@colors@lst@comments}\scshape}%
  }
}
```

We get (exemplarily in Java):

```
1 // Before redefining the style
2 public static void main(...){ /* ... */ }
```

```
1 // After redefining the style
2 public static void main(...){ /* ... */ }
```

Multiple styles can be defined in one go, separated by commas, and survive groups.

## 2 Commands and Environments

Note that the prefix `env@` only indicates the nature of an environment and is not to be attributed to the actual identifier!

### 2.1 The Setting Environments

These environments are created for every loaded language. Every language contained in this package follows the rule that it begins with an “l” (for language) followed by an uppercase letter. So if, for whatever reason, you want to use `java` manually in `env@lstlisting` or similar, the language here is called `lJava`.

▷ `env@<Language>[Style definitions]`

Sets the enclosed code in `<Language>` loaded via `\solLoadLanguage`<sup>→P. 7</sup>. So, important: `env@<Language>` itself does not exist.. Example:

```
1 \begin{java}
2 // A comment!!
3 public static void main(...){ /* ... */ }
4 \end{java}
```

Yields (Meta comment: the display of the  $\text{\LaTeX 2}_{\epsilon}$  code was carried out with `env@latex`):

```
1 // A comment!!
2 public static void main(...){ /* ... */ }
```

Style definitions, by the way, are definitions for listings.

- ▷ `env@<Language>*[Style definitions]`

Analogous to `env@<Language>→P. 5`, but without line numbers.

- ▷ `env@plain<Language>[Style definitions]`

Analogous to `env@<Language>→P. 5`, but without line numbers and background.

- ▷ `\c<Language>[Style definitions]{Code}`

Sets the Code on one line in the respective language. Important note: since  $\LaTeX 2_{\epsilon}$  cannot be prevented from at least simply expanding the characters, characters such as a quotation mark or curly braces must be escaped using a backslash. This can further cause blank fields to be lost in this case. Such a blank field can be forced via “:ws:”, a *literal*.

Example:

This is a test for `\cjava{public static void main}` – did it work?

Yields:

This is a test for `public static void main` - did it work?

- ▷ `\b<Language>[Style definitions]{Code}`

Analogous to `\c<Language>→P. 6`, but no background and no frame are set, which means line breaks are also not a problem.

- ▷ `\i<Language>[Style definitions]{File path}`

Includes the file at `File path` into the document, using `env@<Language>→P. 5` for highlighting.

## 2.2 Language Management

- ▷ `\solStyles`

Shows the formatting and color of the various styles and is probably only useful for testing purposes. So `\solStyles` yields: basic `keywordA` `keywordB` `keywordC` `keywordD` `numbers` `linenumbers` `literals` `comments` *highlight*

- ▷ `\solLanguageSearchPath`

This command contains all the paths in which `\solLoadLanguage→P. 7` should search for languages. This is unnecessary if the language is located in the system's `texmf` tree. The paths can be updated via `\renewcommand`. Example:

```
\renewcommand{\solLanguageSearchPath}{\FolderA/\FolderB/SubfolderA/}{/home/}}
```

The local directory in which `pdflatex` is invoked is always searched! Should there be several files with the same name, the file in the local directory has the highest priority, and afterwards the folders are searched in order until the file is found for the first time.

▷ `\solLoadLanguage{LanguageName}`

Searches for and loads languages in `solLanguageSearchPath`. Several languages can be specified by comma-separating them. The command can be called multiple times, but does not allow the same language to be loaded multiple times. The language must be defined in a file with the naming scheme `language_<LanguageName>.cfg`. So, for example, `language_java.cfg`. `\RegisterLanguage`<sup>→P. 7</sup> explains what the definition of a language has to look like. The language `lVoid` with `void` is always loaded and can be used as a verbatim environment.

▷ `\RegisterLanguage[Aliases]{Language}{LanguageName}`

Registers a language named `LanguageName` as `Language`. This call must be performed by each configuration itself.

In the following example, the language `Rubberduck` is to be defined and loaded. In the configuration file we write:

```
1 \lstdefinlanguage{lRubberduck}{
2   comment=[1]{\#},
3   morekeywords = {Quack, new},
4   morekeywords = [2]{Duck}
5 }
6 \RegisterLanguage{rubberduck}{lRubberduck}
```

So, a language that uses a hash for comments, has the two key terms `Quack` and `new`, and also a “keywordB” (well, the language is not exactly formal anyway :)): `Duck` In the document we load the language with:

```
1 \solLoadLanguage{rubberduck}
```

Now we can also use environments like `env@<Language>`<sup>→P. 5</sup> with `rubberduck`:

```
1 \begin{rubberduck}
2 Duck jens = new Duck(); # A new duck
3 Quack jens ::{
4   Quack Quack, Quack Quack
5   Quack Quack. # Duckish, it's that simple
6 }
7 \end{rubberduck}
```

Yields:

```
1 Duck jens = new Duck(); # A new duck
2 Quack jens ::{
3   Quack Quack, Quack Quack
4   Quack Quack. # Duckish, it's that simple
5 }
```

Commands such as `\c<Language>`<sup>→P. 6</sup>, i.e. `\crubberduck`, of course also work: `Duck jens`.

If `fakeminted`<sup>→P. 4</sup> is not set, the aliases currently have no meaning. If no aliases are otherwise passed, a mapping (`\solmintedmap`<sup>→P. 10</sup>) from `Language` to `LanguageName` is automatically created. Otherwise, a mapping to `LanguageName` is created for each (comma-separated) alias, so that it can be used with `env@minted`<sup>→P. 10</sup>.

## 2.3 General Commands

▷ `\thesolversion`

Returns the current version of the package. So `\thesolversion` yields: `v1.0.23`

Note: via `\value{solversion}` the version can be obtained as a 4-digit number: `1023`.

▷ `\solSetLeftMargin{Length}`

Sets the spacing of the left side of the listing and can thereby ensure that, for example, the line numbers also do not protrude beyond the document margin. Example:

```
1 \solSetLeftMargin{15pt}
2 \begin{java}
3 // A comment!!
4 public static void main(...){ /* ... */ }
5 \end{java}
```

Yields:

```
1 // A comment!!
2 public static void main(...){ /* ... */ }
```

▷ `\solSetRightMargin{Length}`

Sets the spacing of the right side of the listing, otherwise analogous to `\solSetLeftMargin` → p. 8.

▷ `\solSetNumSep{Length}`

Sets the distance between the line numbers and the code.

▷ `\solSetFrameRule{Length}`

Sets the thickness of the border (the default here is 0.75pt, so don't overdo it :P).

▷ `\T{Text}`

Sets the Text in typewriter font.

▷ `\solGetStyle{StyleName}`

Expands to the style of StyleName as used by the package itself.

▷ `\solGet{StyleName}{Text}`

Sets Text in the style of StyleName as used by the package itself. For example:

`\solGet{comments}{Hello World}` yields: Hello World.

▷ `\solNumFs{size}`

Changes the font size of the line numbers separately.

▷ `\lstfs⟨*⟩[linespread]{size}`

Adjusts the font size of the code as well as that of the line numbers. For the latter, an attempt is made to find a suitable size. The starred variant has *no* specification of size; in this case, the current font size is set automatically.

```

1 \begin{java}
2 public static int add(int a, int b){ return a + b; }
3 \end{java}
4 {\lstfs{5}
5 \begin{java}
6 public static int add(int a, int b){ return a + b; }
7 \end{java}
8 }
9 \begin{java}
10 public static int add(int a, int b){ return a + b; }
11 \end{java}
12 {\tiny
13 \begin{java}
14 public static int add(int a, int b){ return a + b; }
15 \end{java}
16 }

```

Ergibt:

```
1 public static int add(int a, int b){ return a + b; }
```

```
1 public static int add(int a, int b){ return a + b; }
```

```
1 public static int add(int a, int b){ return a + b; }
```

```
1 public static int add(int a, int b){ return a + b; }
```

▷ `\lstcolorlet{lstcolor}{normalcolor}`

Assigns a color via `\colorlet`. Here, *lstcolor* may only take one of the following values: *keywordA*, *keywordB*, *keywordC*, *numbers*, *literals*, *comments*, *highlight* or *command*.

▷ `\lstcolordef[mode=RGB]{lstcolor}{colorcode}`

Assigns a color via `\definecolor`. Here, *lstcolor* may only take one of the following values: *keywordA*, *keywordB*, *keywordC*, *numbers*, *literals*, *comments*, *highlight* or *command*. The first argument specifies the mode according to *xcolor*.

▷ `\solguard{Lines}`

With the package option `guardspace` (`noguardspace`)<sup>→ p. 3</sup>, this can be used to specify how many lines need to be protected. The default is: “.8”. Note: this only concerns the lines for the beginning. The last line is currently always secured with `\nobreak`.

▷ `\solblacklistlinenumbers{*}{LineNumbers}`

A comma-separated list of line numbers which should not be displayed. The blocking occurs locally; the blacklisting can thus be controlled via  $\text{\TeX}$  groups. With `\solblacklistisghost`<sup>→ p. 10</sup> `\solblacklistispresent`<sup>→ p. 10</sup> it can be controlled whether the hidden line numbers should be removed from the count. This can also be regulated via the optional star (`\solblacklistisghost`<sup>→ p. 10</sup> is the default; with a star, `\solblacklistispresent`<sup>→ p. 10</sup> is executed instead). Repeated calls add the lines to be hidden to the existing ones.

```

1 \solblacklistlinenumbers{3}
2 \begin{java}
3 public static void main(String[] args){
4     System.out.println("Hello World");
5     System.out.println("Ghooost!");
6 }
7 \end{java}

```

Yields:

```

1 public static void main(String[] args){
2     System.out.println("Hello World");
3     System.out.println("Ghooost!");
4 }

```

In contrast, with a star:

```

1 \solblacklistlinenumbers*{3}
2 \begin{java}
3 public static void main(String[] args){
4     System.out.println("Hello World");
5     System.out.println("Ghooost!");
6 }
7 \end{java}

```

Yields:

```

1 public static void main(String[] args){
2     System.out.println("Hello World");
3     System.out.println("Ghooost!");
4 }

```

#### ▷ \solblacklistisghost

Lines hidden with `\solblacklistlinenumbers`<sup>→ p. 9</sup> are also not counted.

#### ▷ \solblacklistispresent

Lines hidden with `\solblacklistlinenumbers`<sup>→ p. 9</sup> are nevertheless counted.

## 2.4 Minted-Specific

If the package option `fakeminted`<sup>→ p. 4</sup> is set, the following commands and environments also exist.

#### ▷ env@minted[*listing-args*]{language}

Sets the enclosed text in the style of `\solsetmintedstyle`<sup>→ p. 11</sup>, with the language mapped as configured in `\solmintedmap`<sup>→ p. 10</sup>.

#### ▷ \solmintedmap{from}{to}

Creates or overwrites a language mapping for `env@minted`<sup>→ p. 10</sup>. This is done automatically by `\RegisterLanguage`<sup>→ p. 7</sup> (though directly through internal mechanisms).

▷ `\solsetmintedstyle{default/nonumber/plain/plain number}`

Updates the style to be used by `env@minted`  $\rightarrow$  p. 10.

## A Language Definition

### A.1 Predefined Languages

The language files defined here are located in the subfolder Languages and can also be used only as a basis for your own language definition. They already contain a few example values, as I use them, but can in that regard also be completely reset to “Blank”. Note: the definition of languages proceeds, surprise, analogously to Lilly (<https://github.com/EagleoutIce/LILLY>, only the required file naming and the further options differ.), so these language definitions can also be used as an example. Further information is provided by the documentation of listings.

*Java (lJava, java)*

Defines nothing special in this context.

*Bash (lBash, bash)*

Defines nothing special in this context.

*Json (lJson, json)*

Defines nothing special in this context.

*latex (lLatex, latex)*

Defines command as an additional style for commands.

*XML (lXml, xml)*

Defines nothing special in this context.

*Rubberduck (lRubberduck, rubberduck)*

Defines nothing special in this context. Not a real language either :P.

### A.2 Literates

In the context of various programming languages, the wish soon arose to be able to set various symbols correspondingly easily. The replacement rules are loaded by default when the package is loaded, but individual programming languages (such as Java with its escape sequences) can define their own on top. The replacement rules make it possible to include not only umlauts but also symbols. The replacement rules are not handled via a list and are as varied as the needs require. Below is a listing of all the important replacement rules:

|                            |                            |                           |                            |
|----------------------------|----------------------------|---------------------------|----------------------------|
| <code>:bs: “\”</code>      | <code>:ws: “ ”</code>      | <code>:float: “f”</code>  | <code>:bcmd: “\”</code>    |
| <code>:bmath: “\$”</code>  | <code>:cdots: “...”</code> | <code>:exp: “e”</code>    |                            |
| <code>:emath: “\$”</code>  | <code>:cdot: “.”</code>    | <code>:yields: “→”</code> | <code>:star: “*”</code>    |
| <code>:dollar: “\$”</code> | <code>:ldots: “...”</code> | <code>:lan: “⟨”</code>    |                            |
| <code>:space: “ ”</code>   | <code>:c: “”</code>        | <code>:ran: “⟩”</code>    | <code>:percent: “%”</code> |

Note: yes, `:c:` expands to nothing. It can be used as a separator. However, this should not be necessary, since from version 1.0.14 onward the package takes automatic measures: `14 + 3 = 17 * (4 - 3) → $ 17$... !`

## B Examples and Tests

The code for this can be found at the end of the documentation's source code:

So: `public static void main(...)` defines the main method... Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

```
// This is very good code
```

```
public static void main(String[] args){
    System.out.println("Hello World");
    System.out.print(2+3*4-5)
}
```

```
1 import java.util.Scanner;
2
3 /**
4  * @file ean.java
5  * @author Florian Sihler, Raphael Straub
6  * @brief The program calculates the corresponding checksum from the first 12 digits of an EAN
7  * @version 1.0
8  * @date 2.11.2018
9  */
10
11 /**
12  * @class ean
13  * @brief The main class of the program; the entire code 'lives' in it
14  */
15 public class ean {
16     ///@brief the entry point of the program
17     public static void main(String[] args){
18         Scanner scanner = new Scanner(System.in); //Initialization
19         int sum = 0; boolean toggle = false;
20         for(int i = 0; i < 12; i++){ //Reading in the 12 numbers
21             /* alternating *1 and *3 */
22             sum += (scanner.nextInt() * ((toggle)?3:1)); //Summing the number
23             toggle = !toggle; //Inverting the multiplier
24         }
25         System.out.println( (10 - (sum % 10)) % 10 );
26         // Outputs the checksum as the distance to the next multiple of 10
27         scanner.close();
28     }
29 }
```

```
<?xml version="1.0" encoding="UTF-8"?>
<collection>
  <book category="Novel">
    <!-- I am a comment -->
    <title>Great Book</title>
    <authorlist>
      <author>Nice Person</author>
      <author>Even Nicer Person</author>
    </authorlist>
    <description />
  </book>
</collection>
```

```
    <date> today, when else? </date>
  </book>
  <book category="Good_Books">
    ...
  </book>
</collection>
```

```
1 {"menu": {
2   "id": "file",
3   "value": "File",
4   "popup": {
5     "menuitem": [
6       {"value": "New", "onclick": "CreateNewDoc()"},
7       {"value": "Open", "onclick": "OpenDoc()"},
8       {"value": "Close", "onclick": "CloseDoc()"}
9     ]
10  }
11 }}
```

```
1 Just a plain verbatim output. 1234 Hello42, Hello 42, Hello 42
```